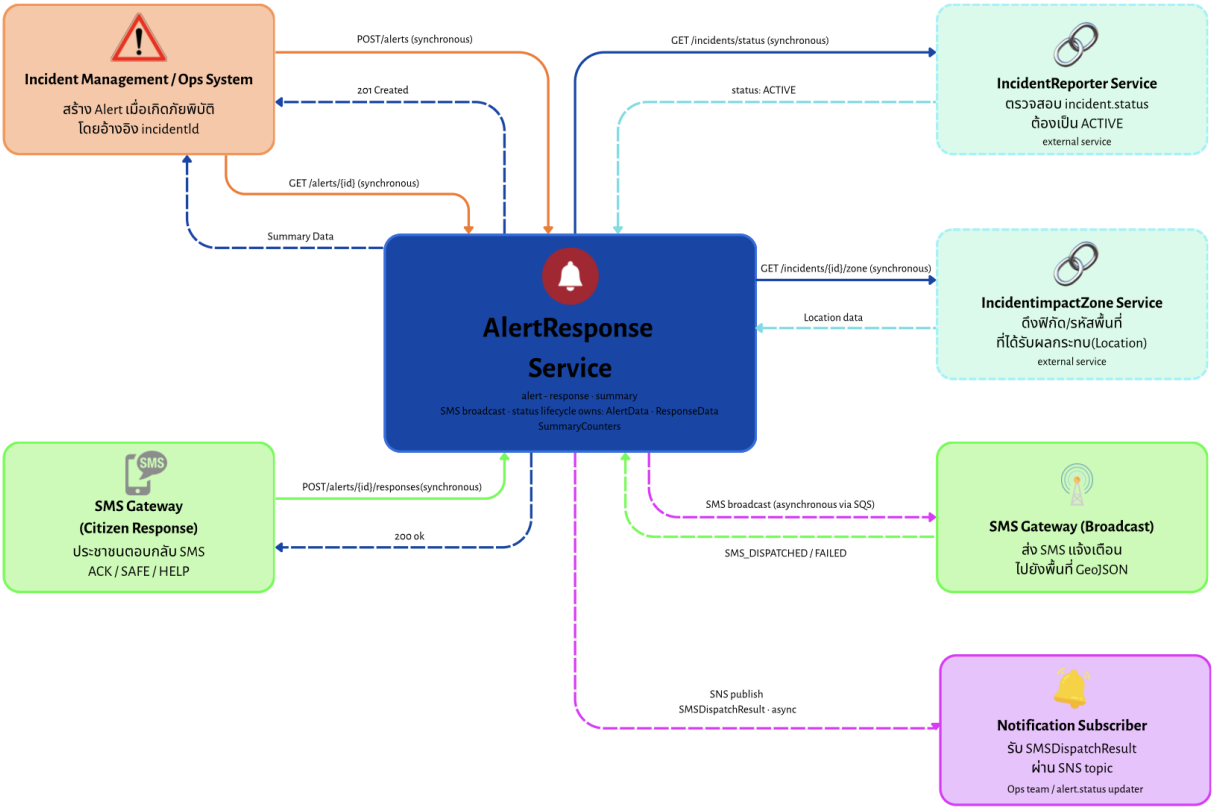
Service Interaction



Service Interaction (การโต้ตอบระหว่างบริการ)

ระบบ AlertResponse Service ของเราทำงานเป็นตัวกลางที่ต้องเชื่อมต่อกับหลายๆ องค์ประกอบครับ โดยสามารถแบ่งทิศทางการสื่อสารออกเป็น 2 ฝั่งหลักๆ คือ **Upstream** (ฝั่งที่เรียกเข้ามาหาเรา) และ **Downstream** (ฝั่งที่เราเรียกออกไป) ดังนี้ครับ:

Upstream Services (บริการต้นทางที่เรียกเข้ามาหาระบบเรา)

เป็นกลุ่มที่เรียกใช้งานเข้ามาที่ API ของเราโดยตรง ซึ่งเราออกแบบให้ตอบกลับทันที (Synchronous) เพื่อประสบการณ์ใช้งานที่ดีครับ

1. **ระบบจัดการของเจ้าหน้าที่ (Operator / Admin UI):**
    - เข้ามาเรียกใช้ API แบบ **Synchronous** ผ่าน `POST /alerts` เพื่อสั่งสร้างการแจ้งเตือนภัย ระบบเราจะรับเรื่องและตอบสถานะ `201 Created` กลับไปให้ทันทีโดยไม่ต้องรอส่ง SMS จนเสร็จ เพื่อป้องกันไม่ให้ค้าง
    - นอกจากนี้ยังเรียก `GET /alerts/{id}` แบบ **Synchronous** เพื่อดึงตัวเลขสถิติแบบ Real-time (ยอดคนปลอดภัย/ขอความช่วยเหลือ) ไปแสดงผลบน Dashboard
2. **SMS Gateway (ช่องทางรับข้อความจากประชาชน):**
    - เมื่อประชาชนในพื้นที่เสี่ยงพิมพ์ SMS ตอบกลับมา Gateway จะยิง Request เข้ามาที่ `POST /alerts/{id}/responses`แบบ **Synchronous** เพื่อให้ระบบเราตรวจสอบข้อมูล และทำการนับยอดสรุป (Atomic Update) ลงฐานข้อมูลทันที

Downstream Services (บริการปลายทางที่เราเรียกใช้งานหรือส่งงานต่อ)

ส่วนนี้คือจุดที่ระบบของเราต้องไปดึงข้อมูลหรือโยนงานไปให้คนอื่นทำต่อ **ซึ่งครอบคลุมการเชื่อมต่อกับ Service ของเพื่อนร่วมชั้นเรียน (Classmate Services) ถึง 2 บริการครับ**:

1. **IncidentReporter Service (Classmate Service):**
    - ทันทีที่มีคำสั่งสร้างการแจ้งเตือน ระบบเราจะวิ่งไปเรียก API ของเพื่อนตัวนี้แบบ **Synchronous** เพื่อดึงข้อมูล "สถานะ (Status)", "ความรุนแรง (Severity)" และ "ประเภทภัยพิบัติ (Incident Type)" มาประกอบร่างเป็นข้อความแจ้งเตือนที่สมบูรณ์และแม่นยำ
2. **IncidentImpactZone Service (Classmate Service):**
    - ในจังหวะเดียวกัน เราก็วิ่งไปเรียก API ของเพื่อนตัวนี้แบบ **Synchronous** ด้วย เพื่อขอดู "พิกัดหรือรหัสพื้นที่ (Location)" ที่กำลังเกิดภัยพิบัติ ข้อมูลนี้สำคัญมากเพราะเราต้องใช้เพื่อระบุขอบเขตว่าต้องดึงเบอร์โทรศัพท์ของประชาชนในโซนไหนบ้างมาทำการส่ง SMS
3. **SMS Gateway (ช่องทางกระจายข้อความเตือนภัย):**
    - เนื่องจากงานส่ง SMS ทีละหลายพันเบอร์ใช้เวลานาน ระบบเราจึงไม่ทำเองแบบรอผล แต่ใช้วิธีโยนงานเข้าคิว SQS แล้วให้ Lambda Worker ทยอยส่งคำสั่งไปยัง SMS Gateway แบบ **Asynchronous** (ทำงานอยู่เบื้องหลัง) ซึ่งตัว Worker มีระบบดักจับข้อผิดพลาดและ Retry ให้รวดอัตโนมัติหากเครือข่ายมือถือมีปัญหา
4. **ระบบประกาศผลแจ้งเตือน (ผ่าน Amazon SNS):**
    - เมื่อ Worker กระจาย SMS เสร็จสิ้น จะไม่ไปยุ่งกับ Database เอง แต่จะประกาศผลลัพธ์ออกไปทาง SNS Topic แบบ **Asynchronous** เพื่อให้ Subscriber ที่รอฟังอยู่มารับช่วงต่อไปอัปเดตสถานะของ Alert ว่าสำเร็จ (ACTIVE) หรือล้มเหลว (FAILED)

Summary (สรุปภาพรวมการโต้ตอบ)

ระบบของเรามีการผสมผสานระหว่างการทำงานแบบ Sync และ Async ได้เลือกใช้ **Synchronous** ในจุดที่ต้องการความถูกต้องและข้อมูลที่เป็นปัจจุบันทันที (เช่น การตรวจสอบพิกัดความรุนแรงจาก Service ของเพื่อน และการรับการโต้ตอบขอความช่วยเหลือจากประชาชน) ในขณะเดียวกัน เราเลือกใช้ **Asynchronous (SQS/SNS)** ในงานที่ใช้เวลานานและกินทรัพยากรสูง (เช่น การกระจาย SMS จำนวนมาก) เพื่อทำ Decoupling ลดภาระของระบบหลัก ทำให้ระบบสามารถสเกลรองรับสถานการณ์ฉุกเฉินที่มีโหลดสูงๆ ได้อย่างมั่นคงครับ